

Domain 3

Claude Code Configuration & Workflows

20% of exam · 6 task statements

3.1 CLAUDE.md hierarchy · 3.2 Custom commands & skills · 3.3 Path-specific rules · 3.4 Plan mode · 3.5 Iterative refinement · 3.6 CI/CD integration

3.1 CLAUDE.md Hierarchy, Scoping & Modular Organisation

Persistent project memory for Claude Code

What is CLAUDE.md?

A markdown file that provides persistent project memory and instructions to Claude Code. It tells Claude about codebase conventions, build commands, test patterns, and code style rules. It is read automatically at session start — no manual injection required.

The 3-level configuration hierarchy

User-level**~/.claude/CLAUDE.md**

Personal preferences only. NOT shared via version control. Teammates never see this. e.g. personal editor preferences, tone, shortcuts.

Project-level**.claude/CLAUDE.md or root CLAUDE.md**

Shared with the whole team. Committed to version control. Primary location for build commands, code standards, test patterns, architecture notes.

Directory-level subdirectory CLAUDE.md

Scoped to that directory and below. Activated when Claude works in that path. e.g. packages/api/CLAUDE.md only loads when Claude is in that package.

KEY CONCEPT

Key distinction: user-level (~/.claude/CLAUDE.md) is personal and NEVER shared. Project-level is committed and shared with all teammates. If a new team member isn't following conventions, the fix is almost always: move instructions from user-level to project-level.

CLAUDE.md content — what belongs in it

```
# Build & Test Commands
- Run tests: `npm test`
- Build: `npm run build`
- Lint: `npm run lint`

# Code Style Guidelines
- Use TypeScript for all new utilities
- Always handle API exceptions with try/catch
- Prefix helper functions with descriptive comments

# Architecture Notes
- Services live in /src/services/
- Never call the DB directly from controllers
```

Modularising CLAUDE.md — 2 approaches

@import syntax • Reference external files from CLAUDE.md • @import ./standards/typescript.md • Each package imports only the standards relevant to its maintainers • Keeps root CLAUDE.md lean and focused	.claude/rules/ directory • Alternative to a monolithic CLAUDE.md • One file per topic: testing.md, api-conventions.md, deployment.md • YAML frontmatter for path-scoping (see 3.3) • Easier to maintain and review than one huge file
---	--

/memory command — diagnosing configuration issues

Run /memory inside Claude Code to see exactly which memory files are loaded for the current session. Use this when Claude is ignoring instructions or behaving inconsistently — it reveals whether the right CLAUDE.md files are being picked up.

EXAM TRAP — CLAUDE.MD SCOPING

A new teammate doesn't get coding conventions. Root cause: the team lead put instructions in ~/.claude/CLAUDE.md (user-level, personal, never shared). Fix: move to project-level .claude/CLAUDE.md and commit to version control.

3.2 Custom Slash Commands & Agent Skills

Reusable task automation with project and user scoping

Custom slash commands

Reusable, parameterised commands that execute a predefined Claude Code task. Created as markdown files. Invoked with /command-name inside Claude Code. Dramatically reduce repetitive prompt writing across a team.

Project-scoped vs user-scoped commands

Project-scoped (shared) • Stored in .claude/commands/ • Committed to version control • Available to all teammates on the project • e.g. /review-pr, /generate-tests, /update-docs	User-scoped (personal) • Stored in ~/.claude/commands/ • Personal only — not shared with team • Available across all projects for this user • e.g. personal debug helper, preferred summary format
--	---

Command anatomy — example file

```
# .claude/commands/review-pr.md

Review the following pull request for:

1. Logic errors and edge cases
2. Missing error handling
3. Performance concerns
4. Test coverage gaps

Flag each issue with: SEVERITY (critical/warning/info),
FILE, LINE, and a specific recommendation.

PR diff: $ARGUMENTS
```

The \$ARGUMENTS placeholder receives anything typed after the command: /review-pr feature/auth-refactor

Agent Skills vs Slash Commands

Slash command • Runs in the main conversation context • All tool calls visible in main thread • Verbose output stays in main window • Good for quick, simple tasks	Agent Skill (with SKILL.md) • Runs in isolated sub-agent context (fork) • Verbose tool calls stay out of main window • Returns only a clean summary to main context • Good for complex multi-step tasks
EXAM TRAP — COMMANDS SCOPING	A team's custom slash commands aren't visible to new teammates. Root cause: stored in <code>~/.claude/commands/</code> (user-scoped, personal). Fix: move to <code>.claude/commands/</code> and commit to version control.

3.3 Path-Specific Rules

YAML glob frontmatter for file-type scoped instructions

Path-scoped rules with YAML frontmatter

Rules files in `.claude/rules/` can include YAML frontmatter with a glob pattern to activate only when Claude is working on files matching that pattern. This prevents irrelevant rules from consuming context window space.

```
---
glob: "**/*.test.ts"
---

# Testing rules (only active when editing test files)
- Use describe/it blocks with descriptive names
- Each test should assert exactly one behaviour
- Mock all external API calls
- Target 100% branch coverage for utility functions
```

Directory-level CLAUDE.md vs path-scoped rules — critical difference

Directory-level CLAUDE.md • Activated by LOCATION — Claude must be working in that directory or below • <code>packages/api/CLAUDE.md</code> only loads when Claude is in <code>packages/api/</code> • Location-bound — doesn't apply across the codebase • Good for: package-specific context (unique dependencies, API patterns)	Path-scoped rules in <code>.claude/rules/</code> • Activated by FILE PATTERN — applies wherever the glob matches across the entire codebase • <code>glob: "**/*.test.ts"</code> loads for any test file, anywhere in the project • Codebase-wide — not location-bound • Good for: file-type conventions (test rules, migration rules)
KEY CONCEPT	Critical distinction: directory <code>CLAUDE.md</code> is location-bound. Path-scoped rules in <code>.claude/rules/</code> with glob frontmatter apply across the ENTIRE codebase based on file pattern — regardless of which directory Claude is currently working in.

When to use each approach

Use case	Approach	Why
API package has unique conventions	Directory-level <code>CLAUDE.md</code> in <code>packages/api/</code>	Location-specific — only loads when working in that package
Test file coding standards	Path-scoped rule: <code>glob: "**/*.test.ts"</code>	File-type-specific — loads wherever test files are
Team-wide build & test commands	Project-level <code>CLAUDE.md</code>	Always loaded for all team members, all contexts
Personal editor preferences	User-level <code>~/.claude/CLAUDE.md</code>	Personal only — never shared with team

3.4 Plan Mode vs Direct Execution

When to explore before committing

What is Plan Mode?

A two-phase execution model. Claude first produces a written implementation plan for the user to review and approve BEFORE any code is written or files are modified. Only after approval does Claude execute. It is a built-in human-in-the-loop checkpoint.

When to use each — the decision rule

Use Plan Mode when... • Multi-file changes or broad refactoring • Architectural decisions with wide impact • The correct approach needs exploring before committing • Database migrations or schema changes • New feature implementation spanning multiple modules • Any task where reversing mistakes would be costly	Use Direct Execution when... • Clear, well-scoped, single-file tasks • Small fixes or formatting changes • Generating boilerplate from a known template • Tasks where the right approach is obvious • CI/CD pipelines — plan mode requires interactive approval
KEY CONCEPT	Memory aid: plan mode = explore before commit. Use it when the cost of getting it wrong is high, or the correct approach needs human judgment before execution. Direct execution = you already know what to do, just do it.

Plan Mode — 4-stage lifecycle

1	Claude analyses the task Reads relevant files, understands the codebase structure, identifies what needs to change.
2	Claude writes the plan Lists exactly which files will be modified, what changes will be made, and in what order. No code written yet.
3	User reviews and approves Can edit the plan, reject it entirely, or approve it. This is the human-in-the-loop checkpoint.
4	Claude executes Implements the approved plan. If reality diverges from the plan, Claude flags it for re-review.
EXAM TRAP — PLAN MODE	'Add plan mode to reduce false positives in CI code reviews.' Wrong on two counts: (1) plan mode requires interactive user approval — it physically cannot run in a non-interactive CI pipeline. (2) false positives are a prompt specificity problem, not an execution mode problem.
EXAM TRAP — PLAN MODE	Choosing plan mode for a clear, well-scoped single-file fix. Overkill — adds unnecessary approval friction. Direct execution is correct for obvious, low-risk tasks.

3.5 Iterative Refinement Techniques

Self-evaluation loops and multi-instance review

What is iterative refinement?

A workflow pattern where Claude produces an initial output, evaluates it against explicit criteria, identifies gaps or issues, and improves it in a loop — until quality criteria are met. Not a single-shot generation followed by manual human editing.

The 4-stage refinement loop

1	Initial generation Claude produces a first draft — code, review, tests, documentation — based on the initial prompt.
2	Self-evaluation Claude checks its own output against the explicit criteria in the prompt: 'Does this meet the stated requirements?'
3	Gap identification Claude identifies what is missing, incorrect, or incomplete — and specifically what needs to change.
4	Targeted improvement Claude improves only the identified gaps. Loop continues until criteria are met or a configured pass limit is reached.

What makes refinement work — explicit criteria

Vague instructions (don't work) • 'Be thorough in your review' • 'Be conservative about flagging issues' • 'Make sure it's high quality' • 'Add comprehensive tests' • None of these are checkable or evaluable	Specific criteria (work) • 'Flag only when claimed behaviour contradicts actual code' • 'Every public function must have a unit test' • 'Critical = data loss risk, warning = edge case, info = style' • 'Each test must assert exactly one behaviour' • Claude can evaluate each criterion as pass/fail
--	---

Multi-instance review architecture

For large tasks (e.g. reviewing a 50-file PR), a single Claude call suffers from attention dilution — shallow coverage of every file rather than deep analysis of any. The solution is multiple Claude instances working in parallel:

Instance	Role	Files covered
Instance 1	Detailed file review	Files 1–10
Instance 2	Detailed file review	Files 11–20
Instance 3	Detailed file review	Files 21–30
Synthesiser	Aggregate + deduplicate findings	All results from Instances 1–3

EXAM TRAP — REFINEMENT

Claude keeps flagging the same non-issues in code reviews. The fix is never 'reduce Claude's sensitivity' or 'lower confidence threshold.' Fix: replace vague instructions with explicit categorical criteria that define exactly what qualifies as a flag-worthy issue — with concrete code examples for each severity level.

3.6 Integrating Claude Code into CI/CD Pipelines

Non-interactive execution, mandatory flags, and session isolation

Running Claude Code non-interactively

Claude Code is designed for interactive developer use, but it can run in fully automated, non-interactive CI/CD pipelines. This requires specific flags to prevent the pipeline from hanging indefinitely waiting for keyboard input that never comes.

The 2 mandatory flags for CI/CD

Flag	What it does	Why required
-p / --print	Runs Claude Code non-interactively — processes the prompt and exits	Without this, the pipeline job HANGS waiting for interactive input — the most common CI failure
--output-format json	Returns structured JSON instead of rendered markdown	Enables programmatic parsing of Claude's output by downstream pipeline steps

```
# CI pipeline invocation pattern

claude -p 'Review this PR diff for critical issues only' \

--output-format json \

< pr_diff.txt
```

Session isolation between CI jobs

Each CI pipeline run should start with a fresh session — no carry-over context from previous runs. This is the default behaviour when using -p. Do NOT use --resume in CI jobs — you want isolation and reproducibility, not continuity. Each run evaluates the current state independently.

Common CI/CD use cases

Use case	What Claude does	Output
Automated PR review	Analyses diff, flags issues by severity with file and line numbers	JSON issue list → posted as PR comment
Test generation	Reads changed files, generates matching test cases	Test files written to repo
Documentation updates	Updates API docs based on code changes	Markdown files committed to repo
Dependency audit	Reviews new dependencies for security / licence issues	JSON report → CI pass/fail gate

Minimising false positives in CI reviews

False positives (flagging valid code as problematic) are a **prompt engineering problem**, not a model sensitivity problem. The fix is always more explicit criteria:

Strategy	Example
Define exact flag categories	Do not use 'potential issues' — list specific issue types with descriptions
Provide severity code examples	Show a concrete code snippet for each severity level (critical/warning/info)
Explicitly state what NOT to flag	'Do not flag style preferences, naming conventions, or subjective design choices'
Categorical criterion	'Flag only when claimed behaviour contradicts actual code behaviour'

EXAM TRAP — CI/CD — THE -P FLAG

A CI/CD pipeline using Claude Code hangs and never completes. Root cause: missing the -p flag. Without it, Claude Code waits for interactive input that never arrives. This is the single most-tested CI/CD fact in Domain 3.

EXAM TRAP — CI/CD — PLAN MODE

Using plan mode in a CI pipeline to improve review quality. Wrong: plan mode requires interactive user approval — it cannot run non-interactively. It causes the pipeline to hang at the approval step.

EXAM TRAP — CI/CD — --RESUME

Using --resume in CI pipeline jobs to carry over context from previous runs. Wrong: CI jobs should be isolated and reproducible. Use fresh sessions (-p without --resume) so each run evaluates the current state independently.

All Domain 3 Exam Traps

Memorise before exam day

EXAM TRAP — CLAUDE.MD SCOPING	A new teammate doesn't follow conventions — instructions are in ~/.claude/CLAUDE.md (user-level, never shared). Fix: move to project-level .claude/CLAUDE.md and commit.
EXAM TRAP — CLAUDE.MD SCOPING	Slash commands work for one developer but not teammates — stored in ~/.claude/commands/ (user-scoped). Fix: move to .claude/commands/ and commit to version control.
EXAM TRAP — PATH-SPECIFIC RULES	Confusing directory-level CLAUDE.md (location-bound) with path-scoped rules in .claude/rules/ (glob-pattern, codebase-wide). Directory CLAUDE.md only loads when Claude is working in that directory. Glob rules load wherever the pattern matches — across the entire codebase.

EXAM TRAP — SKILLS VS COMMANDS	Using a slash command for a complex multi-step task that floods the main context with verbose tool output. Fix: use an Agent Skill — runs in isolated sub-agent fork, returns only a clean summary to the main window.
EXAM TRAP — PLAN MODE IN CI	'Use plan mode to reduce false positives in CI reviews.' Wrong: plan mode requires interactive approval and cannot run in a non-interactive pipeline. False positives are a prompt specificity problem.
EXAM TRAP — PLAN MODE — OVERKILL	Choosing plan mode for a clear, well-scoped single-file fix. Overkill. Direct execution is correct for obvious, low-risk, single-file tasks.
EXAM TRAP — CI/CD — -P FLAG	Missing the -p flag in a CI/CD pipeline — the pipeline hangs indefinitely. -p is mandatory for non-interactive use. Most-tested CI/CD fact in Domain 3.
EXAM TRAP — CI/CD — --RESUME	Using --resume in CI pipeline jobs. Wrong — CI jobs must be isolated for reproducibility. Use fresh sessions with -p (no --resume) so each run evaluates the current state independently.
EXAM TRAP — REFINEMENT — FALSE POSITIVES	Fixing high false-positive rates by 'lowering Claude's sensitivity.' Not a real lever. Fix: replace vague instructions with explicit, checkable criteria and concrete code examples for each severity level.
EXAM TRAP — REFINEMENT — SCALE	Single-pass review of a 50-file PR causing shallow analysis. Fix: split across multiple Claude instances (one per batch of files) then synthesise — not a single call trying to cover everything at once.

Domain 3 complete — 20% of the CCA-F exam. Next: Domain 4 · Prompt Engineering & Structured Output (20%).